

CVP

ai-agent-attack-defense — 防御研究・統制証拠一式

2026-10-06

Cyber Verification Program 申請パッケージ

プロジェクト: `ai-agent-attack-defense` (w-index-m) 目的: Gambit Security レポート (2026-09-22) の AI エージェント攻撃 (Strix / Cairn / Hermes) を題材にした 防御研究と小ツール群。本パッケージは Anthropic [Cyber Verification Program](#) 申請時に提示する「統制・責任ある取り扱い・技術的検証能力」の証拠を 1 冊に束ねたもの。

申請サマリ

- 想定ティア:
 - 個人の防御研究 → **Defense Access**
 - 組織として認可ラボでの攻撃的テスト → **Red Team Access**
- 責任ある取り扱い: 対象は所有/認可物に限定 (README 明記)。netcheck は範囲外を実行前拒否・同意必須・実行ログ。攻撃レシピや安全機構の除去は作成・再構成しない方針を文書化。
- 隔離・運用統制: 認可ラボ・ランブック、隔離ネットワーク設計、攻撃なし mock 検証 (実機確認済み)。
- 技術的検証能力: Strix のローカル実行 PoC (再現手順・パッチ・検証ログ)。

収録ドキュメント

- CVP 適用メモ&ティア対応表 (`cvp-readiness.md`)
- 3ツール横断 防御・検知ノート (`defense-detection-notes.md`)
- Cairn 認可ラボ・ランブック (`cairn-lab/cairn-authorized-lab-runbook.md`)
- Strix ローカル実行バックエンド設計・PoC (`strix-local-backend-notes.md`)

本パッケージは防御・統制の観点に限定し、攻撃の再現手順や安全機構の回避方法は含まない。

Cyber Verification Program (CVP) 適用メモ & 申請準備

Anthropic の [Cyber Verification Program](#) に、本プロジェクト（`ai-agent-attack-defense`：Gambit レポートの AI エージェント攻撃 Strix/Cairn/Hermes を題材にした**防御研究+小ツール**）をどう対応づけ、どのティアを申請するかを整理したもの。

出典ページの要点は末尾「参考」に記載。数値・制度内容は Anthropic の発表に基づく。

1. CVP 3 ティア（要約）

ティア	対象	許可範囲	申請主体	審査
Defense Access	防御（SOC・IR・マルウェア解析・脆弱性検証）	防御業務向けの制限緩和	組織／実績ある個人研究者も可	数日
Red Team Access	認可されたペンテスト・敵対的評価（重要インフラ IT 含む）	認可済みシステムへの攻撃的テスト（物理被害・大規模妨害・ランサム・安全系無効化は常時ブロック）	組織のみ	数週間
Specialized Access	安全重要システム（航空/電力/通信/金融/政府網）のテスト認可組織	ほぼ最小制限	組織（米政府連携が必要）	—

- 全ティアで Claude Opus 5.5 / Sonnet 5.5 / Mythos 5.1 等にアクセス。
- 申請は Anthropic ポータル。本人確認＋必要なセキュリティ統制の証明が要る。データ保持モニタリング必須（Enterprise Frontier Safeguards でゼロ保持オプション予定）。

2. 本プロジェクトの活動 → ティア対応表

活動（本リポジトリ）	性質	対応ティア
<code>vuln_triage.html</code> / <code>scripts/summarize.py</code> （スキャン結果の集計・トリアージ）	防御（脆弱性検証）	Defense
<code>netcheck/</code> （所有資産の確認専用チェック、確認だけ・攻撃なし）	防御	Defense
<code>.github/workflows/security-scan.yml</code> （Semgrep/Trivy/gitLeaks CI）	防御（コード/依存の検証）	Defense
Strix/Cairn/Hermes のコード・アーキテクチャ解析（ <code>security-scan/*-notes.md</code> ）	防御（マルウェア/攻撃ツール解析）	Defense
Strix のローカル実行 PoC（ <code>strix-local-poc/</code> 、良性コードへの静的解析）	防御寄り／研究	Defense
Cairn を認可ラボで実際に走らせる（ <code>cairn-lab/</code> ランブック）	攻撃的テスト	Red Team（組織のみ）
安全重要システム（電力/通信等）への実テスト	安全重要	Specialized（該当時のみ・米政府連携）

読み方：現状の成果物の大半は **Defense** の射程。Cairn/Strix を認可ターゲットへ能動的に走らせる段になって初めて **Red Team**（組織申請）が必要。

3. 申請準備：本リポジトリが示せる「統制・実績」の証拠

CVP 申請では「必要なセキュリティ統制」と（個人 Defense では）脆弱性報告などの実績が問われる。現状の資産の対応：

- 責任ある取り扱いの実績
 - README に「自分が所有・認可した対象にのみ使用」を明記。
 - `netcheck` は許可レンジ外を**実行前に拒否**／同意必須／ローカル限定トークン／実行ログ（`netcheck.log`）。
 - 攻撃ツール解析で**攻撃レシピ**や安全フィルタ除去の再構成は扱わない方針を文書化（Hermes メモ）。
- 隔離・運用統制の設計
 - `cairn-lab/cairn-authorized-lab-runbook.md`：実行前チェックリスト、ネットワーク隔離、egress 制限、使い捨て/スナップショット、監視、禁止事項。
 - `cairn-lab/dispatch_mock_local.yaml`：**攻撃せずにエンジンのみ検証**した構成（本番前スモーク）。
- 技術的検証能力
 - `strix-local-poc/`：バックエンド差し替え+PoC（再現手順・パッチ・検証ログ）。
- （任意）秘密情報管理： `.gitleaks.toml` / `.pre-commit-config.yaml` によるシークレット検出。

申請時は上記を「統制の証明」資料として束ねられる。秘密情報（LLM キー等）はコミットしない運用を継続する。

4. 推奨アクション

1. 個人で始めるなら → **Defense Access** を申請（審査は数日）。
 - 申請材料：本リポジトリ（防御ツール＋解析ノート＋責任ある取り扱いの記録）。
 - 期待効果：脆弱性検証・攻撃ツール解析の精度向上。
 2. 組織として本番ペネテストまで行うなら → **Red Team Access**（組織申請・数週間）。
 - 申請材料：§3 の隔離設計・ランブック・mock 検証を「統制の証明」として提出。
 - 前提：対象の所有/書面認可、隔離ラボ、データ保持要件の充足。
 3. **不変の一線**：Red Team でも物理被害・大規模妨害・ランサム・安全系無効化はブロック対象。本プロジェクトの方針（攻撃レシピ・安全フィルタ除去を作らない）と整合。
-

参考

- [Anthropic — Cyber Verification Program](#)
 - Project Glasswing と旧 CVP を統合した拡張枠組み。
 - Glasswing 実績: 2026/4-7 に 12.9 万件以上の検証済み脆弱性（うち 3.3 万件超が critical/high）。
 - CyScenarioBench: Defense は悪性多段攻撃 50 件中 46 件をブロック／Red Team は 50 件中 34 件を無制限時と同等に完遂。

3ツール横断 防御・検知ノート（Strix / Cairn / Hermes）

Gambit レポートの攻撃連鎖（偵察→侵入→統括/事後）を、**防御側の観測点と対策**に振って整理したもの。攻撃手順のレシピではなく、「何が観測され、どこで止められるか」に焦点を当てる。

出典: [Gambit Security](#) / 各ツール: [Strix](#) / [Cairn](#) / [Hermes](#)

攻撃連鎖の役割分担（本セッションの解析より）

段階	ツール	やること	技術的特徴
偵察・スキャン	Strix	攻撃面の発見・脆弱性探索	Docker サンドボックスで実行、caido プロキシで HTTP 捕捉、nmap/ffuf/playwright 等を同梱
探索的侵入	Cairn	目標 (shell/flag) まで自走探索	黒板サーバ(:8000)+fact-intent グラフ、00DA ワーカーを per-project コンテナで起動 (NET_RAW/NET_ADMIN)
統括・事後	Hermes	キャンペーン統括・事後作業	汎用エージェント、多数のスキル、Telegram/Discord/Slack ゲートウェイ、cron、サブエージェント、7 実行バックエンド

共通の土台：LLM API への継続的な egress（レポートでは OpenRouter 経由、Claude/DeepSeek 等）。これが 3 ツール共通の最も安定した観測点。

段階別：観測できる兆候 → 防御/検知

A. 偵察・スキャン (Strix 的挙動)

観測点：

- ・ 単一ソースからの広域ポート/パス走査、短時間の多数リクエスト、既知スキャナの UA/挙動。
- ・ 送信元が 自動化ブラウザ (playwright/CDP) 由来のフィンガープリント。
- ・ 外向きに LLM API への定期的・バースト的 HTTPS (スキャン判断のたびに推論)。

防御/検知：

- ・ WAF/レート制限、スキャン検知 (多数 404/403・パス列挙)、ボット対策。
- ・ 管理画面・機微エンドポイントの認証必須化+IP 制限/VPN 内側 (netcheck が「管理画面 URL の応答」を確認対象にしているのと表裏)。
- ・ セキュリティヘッダ整備 (CSP/HSTS/XFO/nosniff)。決済ページの CSP は skimmer 注入対策の要。

B. 探索的侵入 (Cairn 的挙動)

観測点：

- ・ 内部に 黒板サーバ(:8000 / FastAPI) と、per-project ワーカーコンテナが多数起動 (イメージ名 cairn-worker-container、NET_RAW/NET_ADMIN 付与)。
- ・ ワーカーからの反復的な試行→観測→再試行パターン (OODA。失敗しても別 intent で粘る)。
- ・ 継続的な LLM API egress と、対象への断続的なプローブ。

防御/検知：

- ・ EDR/コンテナランタイム監視：NET_RAW/NET_ADMIN 付きコンテナの新規起動、未知イメージの pull、host ネットワークモードのコンテナを警戒。
- ・ 内部セグメンテーション (lateral movement 阻止)、重要資産への最小権限・多要素。
- ・ 送信ドメイン allowlist (LLM API への常時 egress は正規業務でなければ異常)。

C. 統括・事後 (Hermes 的挙動)

観測点：

- ・ メッセージングアプリ (Telegram/Discord/Slack/Signal) への外向き接続 = C2 的チャネルになり得る。
- ・ cron/スケジューラによる永続化、サブエージェントの大量スポン、スキルの動的生成/自己改変。
- ・ 「自身の安全フィルタを外す」類のスキル追加 (Gambit が 78/121 の攻撃スキル、うちフィルタ除去を指摘)。

防御/検知：

- ・ 外向きメッセージング API への接続監視 (サーバ/本番ホストからの Telegram 等は強い異常シグナル)。

- 永続化監視（新規 cron/サービス/スタートアップ項目）、プロセス系統の異常（自動化された連続ツール実行）。
 - LLM 利用ガバナンス：安全機構の除去・迂回を図る挙動は方針・契約・CVP の不変の一線（物理被害/大規模妨害/ランサム/安全系無効化はブロック）に照らして遮断・通報。
-

横断的な「効く」対策（優先順）

1. 外向き通信の allowlist — LLM API・メッセージングへの常時 egress は最強の共通シグナル。本番/サーバ系からの当該通信を既定で遮断・監視。
 2. コンテナ/実行環境の可視化 — 特権 cap 付きコンテナ、未知イメージ、host ネットの起動を検知。
 3. 攻撃面の最小化 — 管理画面の隔離、DB/サービスをネット直結にしない（netcheck の RISKY ポート群が示す露出を塞ぐ）。
 4. 決済・機微ページの CSP — skimmer/カード窃取コード注入の防止。
 5. 永続化・自動化の監視 — cron/スケジュール、連続自動ツール実行、サブエージェント大量生成。
 6. ログと相関 — スキャン痕跡+LLM egress+特権コンテナ+外向きメッセージングを時系列で相関させると、単発では見えない「自律エージェント攻撃」が像を結ぶ。
-

本リポジトリ資産との対応

- `netcheck/` : A の露出面 (RISKY ポート、ヘッダ、管理画面応答) を **自分の資産で確認** するための道具。
- `vuln_triage.html` / `scripts/summarize.py` : スキャナ結果を集約し、A/B の修正優先度付け。
- `security-scan.yml` (Semgrep/Trivy/gitleaks) : コード/依存/シークレットの事前防御。
- `cvp-readiness.md` : これらを CVP (Defense/Red Team) の統制証拠として束ねる枠組み。

本ノートは防御・検知の観点に限定し、攻撃の再現手順や安全機構の回避方法は含めない。

Cairn 動作確認ランブック (自分の隔離ラボ・認可済み前提)

`oritera/Cairn` (AGPL-3.0, v0.2.1) を、**自分が所有・認可した、ネットワーク隔離された“やられ環境”**に対して実際に動かすための設計書。

⚠ Cairn は能動的に脆弱性を突いてシェル/権限を奪う実行役。対象を所有しているか、書面で認可を得ている場合のみ実行すること。 本ランブックは `authorized pentest` / 自前ラボ / CTF を想定し、第三者・本番・未認可の対象には一切向けない。

i この Claude セッションの `sandbox` では実行しない (Docker デーモン無し・LLM キー無し・外部通信制限)。実行はあなたの隔離ラボ側で行う。

0. 実行前チェックリスト（すべて YES でなければ実行しない）

- ☐ 対象は**自分が所有**、または**書面の認可**がある（範囲・期間・対象 IP が明記）。
 - ☐ 対象と Cairn ワーカーは**独立したネットワークセグメント**に隔離（下記「ネットワーク隔離」）。
 - ☐ 対象・ワーカーとも**使い捨て or スナップショット済み**で、終了後に復元できる。
 - ☐ 外部通信は **LLM API エンドポイント（と初回のイメージ取得）**だけに限定。対象以外の内部ホスト・インターネットへ到達しない。
 - ☐ 予算・ステップ上限（タスク timeout / worker 数）を**保守的**に設定。
 - ☐ 実行中は**監視下**に置き、いつでも停止できる（`completed_action: stop` で事後調査可）。
-

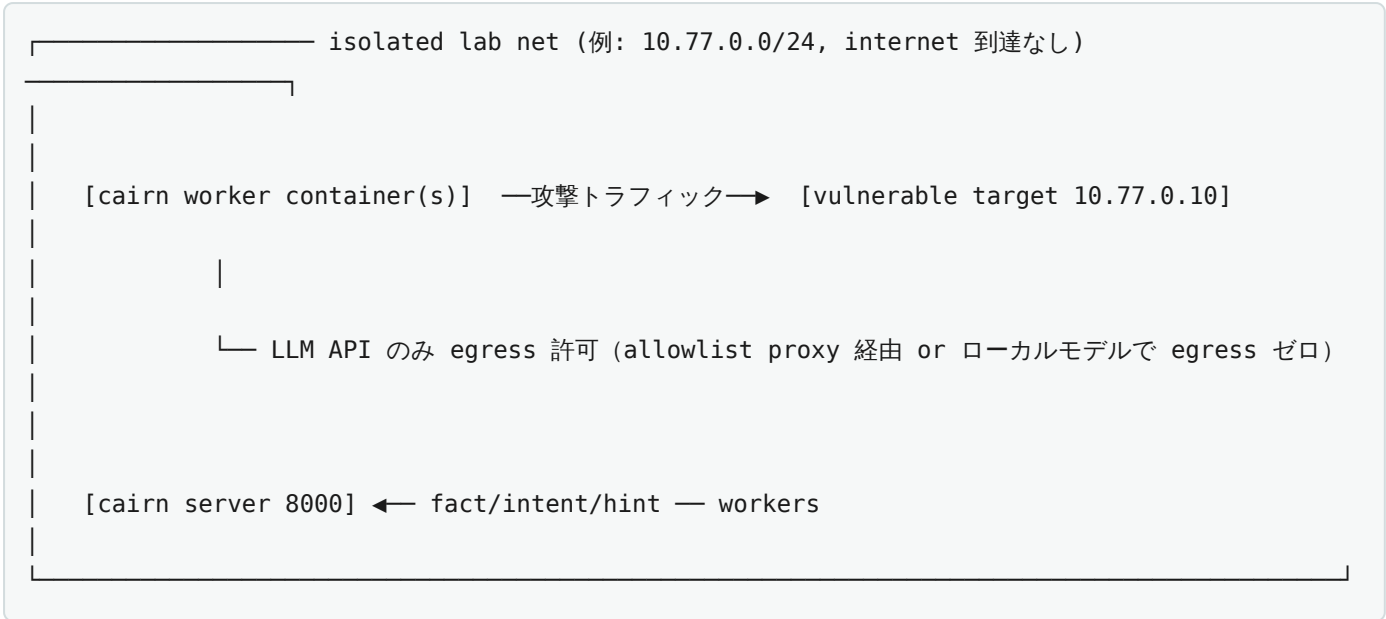
1. 構成要素

要素	役割	実行場所
Cairn Server (cairn serve)	黒板 (fact-intent グラフ) を持つ FastAPI。SQLite に状態保存。 <code>POST /projects</code> で origin/goal を登録	ラボ内のどこでも可 (UI/ API は 8000)
Dispatcher + Workers (cairn dispatch)	スケジューラ+OODA ワーカー。 <code>container</code> モード (<code>ghcr.io/oritera/cairn-worker-container:latest</code> を 1 プロジェクト 1 コンテナで exec、Docker 必要) か <code>local</code> モード (ホストプロセス、 <code>claude/codex/pi</code> CLI を再利用)	隔離ネットワーク上
LLM プロバイダ	ワーカーの推論。 <code>worker env</code> で <code>ANTHROPIC_MODEL</code> / <code>BASE_URL</code> / <code>AUTH_TOKEN</code> (<code>claudecode</code> 型) 等	外部 API or ローカルモデル
やられ対象	origin に記載。例: <code>http://10.77.0.10</code> / goal: <code>get a shell</code> / <code>capture the flag</code>	隔離ネットワーク上

ワーカーは `nmap -sS` など raw socket を使う場合 `cap_add: [NET_RAW, NET_ADMIN]` が要る。付けるのは必要時のみ。

2. ネットワーク隔離の設計（最重要）

`dispatch.example.yaml` は `network_mode: "host"` を例示しているが、**隔離目的には不可**（ホストのネットワーク全体に素通し）。代わりに：



- 専用の **internal** な **Docker ネットワーク**（または隔離 VLAN）を作り、対象とワーカーをそこへ。
- **egress** は **LLM エンドポイント**だけを許可（ファイアウォール allowlist）。可能なら**ローカルモデル**にして egress を無くするのが最も安全。
- 対象以外の内部ホスト、インターネット、会社ネットヘルートが**無い**ことを実行前に確認（`ip route` / 対象外 IP への ping が全て失敗すること）。

3. 手順

3.1 ラボ準備

1. 隔離ネットワークを作成（internet 到達なし）。
2. やられ対象を立てる（自作の脆弱アプリ、または自分が用意した CTF 的ターゲット）。スナップショット取得。
3. Cairn を動かすホスト（ワーカー）もスナップショット可能にする。

3.2 インストール

```
git clone https://github.com/oritera/Cairn && cd Cairn
uv sync --project cairn
# container モードなら worker イメージを取得（この時だけ egress 必要）
docker pull ghcr.io/oritera/cairn-worker-container:latest
```

3.3 設定（dispatch.yaml）

dispatch.example.yaml をコピーして編集。最低限：

- server: をラボ内のサーバ URL に。
- workers[].env に自分の LLM 資格情報（例 ANTHROPIC_MODEL / ANTHROPIC_BASE_URL / ANTHROPIC_AUTH_TOKEN）。
- container.network_mode は host をやめ、作成した隔離ネットワーク名を指定。cap_add は必要時のみ。
- runtime.max_workers / tasks.*.timeout を保守的に（まず小さく）。

秘密情報は dispatch.yaml にベタ書きせず環境変数や secret 管理から注入する。dispatch.yaml はリポジトリにコミットしない。

3.4 まず安全スモーク（mock・攻撃なし）

実標的に向ける前に、エンジンが回ることをモックで確認（§4 参照）。

3.5 本番実行

```
# 1) サーバ
uv run --project cairn cairn serve --host 0.0.0.0 --port 8000

# 2) プロジェクト作成 (origin=ラボ対象, goal=目標) : POST /projects
curl -s -X POST http://<server>:8000/projects \
  -H 'Content-Type: application/json' \
  -d '{"origin":"target http://10.77.0.10","goal":"get a
shell","bootstrap_enabled":true}'

# 3) ディスパッチャ起動 (ワーカーが OODA ループを回し始める)
uv run --project cairn cairn dispatch --config dispatch.yaml
```

3.6 観測・記録

- GET /projects/{id} で fact-intent グラフの成長を確認 (server の static UI でも可)。
- export.py ルータ (/export) で結果を取得し、ラボ記録として保存。
- completed_action: stop なのでワーカーコンテナを事後調査できる。

3.7 片付け

- ディスパッチャ/サーバ停止 → ワーカーコンテナ削除 → 対象とワーカーホストをスナップショットへ復元。
 - egress allowlist を元に戻す。
-

4. 安全スモーク (mock) : 攻撃せずにエンジンだけ回す

`type: mock` ワーカーは LLM も実コマンドも使わず、確率的にダミーの Fact/Intent を出すだけ。

`execution: local` にすればコンテナも不要（ホストの `python3` サブプロセスで動く）。実標的に向ける前の配線確認に最適。

検証済み設定: `dispatch_mock_local.yaml` (`dispatch_mock.yaml` に `runtime.execution: local` + `worker_healthcheck: disabled` を足し、`container:` を除去したもの)。

```
# 依存は標準 PyPI から (Cairn の既定は aliyun ミラー固定なので上書き)
UV_DEFAULT_INDEX="https://pypi.org/simple" uv sync --project cairn

# 1) サーバ
UV_DEFAULT_INDEX="https://pypi.org/simple" uv run --project cairn cairn serve --host
127.0.0.1 --port 8000

# 2) 到達先の無いダミー project (title は必須)
curl -s -X POST http://127.0.0.1:8000/projects -H 'Content-Type: application/json' \
-d '{"title":"mock-smoke","origin":"mock target (no network)","goal":"mock
goal","bootstrap_enabled":true}'

# 3) mock ディスパッチャ (ホストプロセス・攻撃なし)
UV_DEFAULT_INDEX="https://pypi.org/simple" uv run --project cairn cairn dispatch --
config dispatch_mock_local.yaml
```

- fact-intent グラフが OODA で成長していくのを `GET /projects/{id}` / static UI で観察。どこへもパケットは飛ばない。

この環境で検証済み (2026-10-06) : 上記手順で `cairn serve` 起動 → `proj_001` 作成 (origin/goal の 2 Fact) → mock ディスパッチャ起動で、OODA ループが bootstrap→reason→explore を回し、`f001-f003` のダミー Fact と `i001-i005` の Intent を生成。ディスパッチャログに外部 URL/接続は 0 件。Docker/LLM/標的すべて不要で完走した。本番の exploitation はラボ側のみ。

5. やってはいけないこと

- 未認可・第三者・本番環境を `origin` にする。
 - `network_mode: host` のまま、隔離せずに実行する。
 - `egress` を絞らず、対象以外へ到達できる状態で走らせる。
 - 秘密情報を `dispatch.yaml` に残してコミットする。
-

参考（リポジトリ内）

- `dispatch.example.yaml` — container モードの完全な設定例
- `dispatch.local.example.yaml` — local モード（ホスト実行、CLI 再利用）
- `dispatch_mock.yaml` — mock ワーカー（攻撃なし）
- `docs/specs/server-protocol.md` — 黒板プロトコル / `POST /projects` (origin・goal)
- `docs/specs/dispatcher-design.md` — ディスパッチャ設計

Strix をローカル実行する設計メモ（**Docker 不要の local** バックエンド）

対象: `usestrix/strix` v1.7.0 を、Docker デーモンを使わずに ホスト（例: Claude Code の cloud sandbox）上で直接動かす方法の調査メモ。

⚠ Strix は認可されたペネトレーションテスト用ツールです。対象は必ず自分が所有・許可を得たものに限定すること。

背景

Strix は「LLM API キーを入れれば動く」のではなく、スキャン対象を隔離した Docker コンテナを起動し、その中でエージェント（LLM ↔ ツール）を動かす設計。したがって本来の必須要件は 2 つ:

- LLM API キー（`STRIX_LLM` / `LLM_API_KEY`、LiteLLM の model id）
- 稼働中の Docker デーモン

cloud sandbox は Docker-in-Docker を通常持たないため（`docker info` が到達不可）、公式のローカル CLI モードはそのままでは動かない。マネージド版（`app.strix.ai`）はネットワークポリシーで `app.strix.ai` への到達が拒否される場合がある。

実行フロー（コード地図）

```
strix/interface/main.py      エントリ (project.scripts: strix=...main:main)
└─ interface/cli.py run_cli()  引数 → scan_config
    └─ core/runner.py run_strix_scan()
        └─ runtime/session_manager.create_or_reuse() ← サンドボックス起動
            └─ runtime/backends.get_backend(name) ← STRIX_RUNTIME_BACKEND で分岐
                └─ エージェントループ (LLM ↔ tools/*)
```

- `runtime/backends.py` : `STRIX_RUNTIME_BACKEND` (既定 `docker`) でバックエンドを選ぶレジストリ。 `register_backend(name, fn, supports_bind_mounts=...)` で独自バックエンドを追加できる。
- `runtime/docker_client.py` : `openai-agents SDK` の `DockerSandboxClient` を継承し、コンテナに `NET_ADMIN/NET_RAW` (`nmap -sS` 等の raw socket 用)、 `host.docker.internal` ゲートウェイを付与。
- `tools/shell` の `exec_command` は **SDK 提供** (`agents.sandbox.capabilities.tools.shell_tool.ShellTool`)。 エージェントの全コマンド実行はサンドボックスセッション経由 → 「セッション = 実行環境」。

結論：ゼロから書く必要はない

`openai-agents SDK` に `UnixLocalSandboxClient` (ローカル Unix サンドボックス) が同梱されている。

```
.venv/.../agents/sandbox/sandboxes/unix_local.py
class UnixLocalSandboxClient      # create() で session を返す
class UnixLocalSandboxSession    # ホスト上で直接コマンド実行、workspace = manifest.root (ホスト FS)
```

Docker 版 `_docker_backend` の実体は「SDK の Client を作って session を返す」だけなので、これを Unix 版へ差し替えるだけで成立する。

実装案 (新バックエンド ~15 行)

`strix/runtime/backends.py` に追加:

```
async def _unix_local_backend(
    *, image, manifest, exposed_ports, bind_mounts=None,
):
    # image と bind_mounts は使わない (ホスト FS がそのまま workspace)
    from agents.sandbox.sandboxes.unix_local import (
        UnixLocalSandboxClient, UnixLocalSandboxClientOptions,
    )
    client = UnixLocalSandboxClient()
    options = UnixLocalSandboxClientOptions(exposed_ports=exposed_ports)
    session = await client.create(options=options, manifest=manifest)
    await session.start()
    return client, session

# bind mount 非対応 (= manifest entries 経由でソースを materialize)
register_backend("local", _unix_local_backend, supports_bind_mounts=False)
```

起動:

```
export STRIX_RUNTIME_BACKEND=local
export STRIX_LLM="anthropic/claude-sonnet-5-5" # LiteLLM の model id
export LLM_API_KEY=...
uv run strix -n -t ./ --scan-mode quick --max-budget 10
```

`get_backend()` が `STRIX_RUNTIME_BACKEND` で分岐するため、既存の Docker 経路には触れない。

注意点（ここが実質的な作業）

#	論点	内容
①	ツールの非同梱	nmap / ffuf / caido(プロキシ) / playwright 等は Docker イメージ側に同梱。ローカルには無い → 使うツールだけホストに <code>apt</code> / <code>pip</code> で導入が必要。
②	manifest の users/groups 不可	<code>UnixLocalSandboxSession</code> は manifest の user/group 指定を非サポート（コードに明示）。 <code>session_manager._host_identity_env</code> が uid/gid を積むため、local 経路では渡さない調整が要る。
③	caido 前提の除外	Docker 版は起動時に <code>bootstrap_caido()</code> で in-container プロキシ(48080)へ接続。ローカルには無い → caido をホスト起動するか、 <code>proxy</code> ツール依存フローを無効化。
④	分離性の喪失（最重要）	Docker 隔離をやめてホストで直接ツール実行 = 攻撃実行環境がホストになる。対象は自分が権限を持つものに限定。 <code>nmap -sS</code> 等は root/cap が必要。
⑤	bind mount vs materialize	<code>supports_bind_mounts=False</code> なら <code>session_manager.build_manifest_entries()</code> が <code>LocalDir</code> でソースを workspace に展開。 <code>-t ./</code> のローカルコード解析はこれで動く。

推奨スコープ（最小 PoC）

`-t ./`（ローカルコードの静的解析 = `code_review` 相当）に限定すれば、ネットワーク系ツール（nmap/caido）が不要になり、①③④をほぼ回避できる。まずこの構成で PoC するのが現実的。実スキャンには LLM API キーが必要。

確認済みの事実（この環境での動作確認）

- `uv sync` で依存導入、`uv run strix --help` 正常 (v1.7.0)。
- `docker info` 到達不可（ローカル CLI の実スキャンは不可）。
- `strix cloud login` はネットワークポリシーにより `app.strix.ai` へ 403（プロキシ CONNECT 拒否）。
- SDK に `unix_local.py` が存在することを確認。

PoC 結果（実装して検証した）

設計案どおり 2 ファイルにパッチを当て、PoC を実行して成立を確認した。パッチ: `strix-local-poc/strix-local-backend.patch` 検証スクリプト: `strix-local-poc/poc_local_backend.py`

当てたパッチ (3 ファイル)

1. `strix/runtime/backends.py`: `_unix_local_backend` を追加し `_BACKENDS["local"]` に登録。さらに `caido` 対応をバックエンド単位で持つレジストリ `_CAIDO_BACKENDS` (既定 `{"docker"}`) と `backend_supports_caido()` を追加。 `register_backend` に `supports_caido` 引数を追加。
2. `strix/interface/main.py`: Docker 事前チェック (`check_docker_installed` / `pull_docker_image`) を `settings.runtime.backend == "docker"` のときだけ実行するようゲート。
3. `strix/runtime/session_manager.py`: `create_or_reuse` の `caido` 結合をバックエンド単位でゲート。 `caido` 非対応バックエンド (`local`) では (a) `caido` プロキシ env (`http_proxy` / `https_proxy` / `ALL_PROXY`) を注入しない、(b) `exposed_ports=()`、(c) `resolve_exposed_port` と `bootstrap_caido` をスキップ、(d) `caido_client=None`。 proxy ツールは既存の `graceful-degradation` (`_ctx_client` が `None` → 「利用不可」) に乗る。

検証 1: バックエンドを直接叩く (LLM 非依存)

`poc_local_backend.py` を `uv run python` で実行した結果:

- `supported_backends()` → `['docker', 'local']`、`get_backend('local')` 解決 OK。
- `DOCKER_HOST=unix:///nonexistent-docker.sock` にしても **セッション起動成功** (`UnixLocalSandboxSession`)。 Docker デーモンに一切触れない。
- `session.exec(...)` がホスト上で実行される (`uname -a` が実ホスト、`whoami=root`、`python3 --version=3.13.16`)。
- `LocalDir` で対象ソースが `workspace` に `materialize` され、`ls repo` に `netcheck` / `scripts` / `security-scan` / ... が出現。
- 対象リポジトリ自身のツール `scripts/summarize.py` をセッション内で実行し完走 (`exit 0`)。

検証 2: 実 CLI 配線 (`strix -n -t ./ --scan-mode quick`)

`STRIX_RUNTIME_BACKEND=local` で CLI を起動:

- パッチ前: 起動時の Docker 事前チェックで `DOCKER NOT AVAILABLE` により即停止。
- パッチ後: Docker ゲートを通過し、環境検証 → LLM プリフライト接続まで到達。 ダミーキーのため `invalid x-api-key` で停止 (`api.anthropic.com` への到達自体は成功)。 → **残る唯一の前提は有効な LLM API キーのみ**。実キーがあれば検証 1 で実証済みの ローカルセッション起動を経てエージェントループに進む。

検証 3: caido 非依存 (`create_or_reuse` を `local` で直接呼ぶ)

`poc_caido_free.py` を実行した結果:

- `backend_supports_caido('local')=False` / `('docker')=True`。
- `create_or_reuse(backend=local)` が `caido bootstrap` を呼ばずにセッションを返す。
- `bundle["caido_client"] is None` (proxy ツールは「利用不可」に `degrade`。クラッシュしない)。
- セッション内に `caido` プロキシ env が注入されていない (`echo $http_proxy $ALL_PROXY` が空)。
- 対象ソースの `materialize` と `session.exec` は正常。

→ これで `local` バックエンドは `caido` サイドカー無しで完結する。コード解析系スキャン（`proxy` ツールを使わないフロー）なら、追加の外部依存なしに動かせる。

PoC で判明した追加の実挙動（設計メモの補足）

- `base_dir = Path.cwd()` 制約: SDK の `LocalDir` 展開は、ソースが CWD 配下でないと `LocalDirReadError (outside_base_dir)` で拒否する（source grant 未指定時）。Strix の正規の使い方 `-t ./`（対象＝カレント）では常に満たされるため実害なし。CWD 外を対象にする場合は source grant の付与か、対象を CWD 配下に置く必要がある。
- Docker 事前チェックの無条件実行: `main.py` の `check_docker_installed` / `pull_docker_image` はバックデンド設定を見ずに走るため、`local` でも上記ゲートが必須。
- シンボリックリンクは `LocalDir` 展開で非サポート（`symlink_not_supported`）。`.git` などリンクを含むツリーは除外して渡すのが無難。

未検証（本 PoC のスコープ外）

- 実 LLM キーでのエンドツーエンドのスキャン完走。
- 動的スキャン（`live_test`）で `proxy` ツールを使うフロー。`local` では `caido` が無いため トラフィック捕捉系は使えない（コード解析系に限定すれば問題なし）。
- nmap など外部ツールを要する動的スキャン（ホストに未導入）。